

Parul[®]
University

NAAC A++

FACULTY OF ENGINEERING AND TECHNOLOGY
BACHELOR OF TECHNOLOGY

Enterprise Programming Using Java
(303105310)

5th SEMESTER

COMPUTER SCIENCE &
ENGINEERING DEPARTMENT

Lab Manual

CERTIFICATE

This is to certify that Student **Mr. Manish Kumar Patel**, with Enrollment No. **2203051050729** has Successfully completed his Laboratory experiments in **Enterprise Programming Using Java (303105310)** from the department of **Computer Science & Engineering** during the academic year **2024-2025**.



Date of Submission: _____

Staff in Charge: _____

Head of Department: _____

INDEX

Sr. No.	Name of Practical	Page No.	Date of Performance	Date of Submission	Marks	Signature
1.	Write a program to insert and retrieve the data from database using JDBC.					
2.	Write a program to demonstrate the use of PreparedStatement and ResultSet interface.					
3.	Servlet Execution on tomcat					
4.	A servlet program to print hello world					
5.	A servlet program to display request details A servlet program to handle user form					
6.	A servlet program to create a cookie and display cookie					
7.	A servlet program to do session tracking					
8.	Write a program to implement chat Server using Server Socket and Socket class.					
9.	Write a Servlet program to send username and password using HTML forms and authenticate the user.					

10.	JSP program to display hello world.					
11.	JSP program to demonstrate arithmetic operations					
12.	JSP program to demonstrate JSP: forward action tag JSP program to request implicit object					
13.	Create application to store the data in database to perform Hibernate CRUD operations					
14.	Create a application store the data in database to perform Spring CRUD operations					
15.	Create a web application to store the data in database with spring boot.					

Experiment No: 1**Problem Statement:**

Write a program to insert and retrieve the data from database using JDBC

Code:

```
import java.sql.Connection;

import java.sql.DriverManager;

import java.sql.PreparedStatement;

import java.sql.ResultSet;

import java.sql.SQLException;

public class SimpleJdbcExample {

    private static final String URL = "jdbc:mysql://localhost:3306/manishdatabase";

    private static final String USER = "root";

    private static final String PASSWORD = "password";

    public static void main(String[] args) {

        String insertQuery = "INSERT INTO users (name, email) VALUES (?, ?)";

        String selectQuery = "SELECT * FROM users";

        try (Connection connection = DriverManager.getConnection(URL, USER, PASSWORD)) {

            try (PreparedStatement insertStmt = connection.prepareStatement(insertQuery)) {

                insertStmt.setString(1, "Manish Patel");

                insertStmt.setString(2, "maneeshkurmii@gmail.com");

                int rowsInserted = insertStmt.executeUpdate();

                System.out.println("Rows inserted: " + rowsInserted);

            }

            try (PreparedStatement selectStmt = connection.prepareStatement(selectQuery);

                ResultSet resultSet = selectStmt.executeQuery()) {
```

```
while (resultSet.next()) {  
    int id = resultSet.getInt("id");  
    String name = resultSet.getString("name");  
    String email = resultSet.getString("email");  
    System.out.println("ID: " + id + ", Name: " + name + ", Email: " + email);  
}  
}  
} catch (SQLException e) {  
    e.printStackTrace();  
}  
}  
}
```

Output:

Rows inserted: 1

ID: 1, Name: Manish Patel, Email: maneeshkurmii@gmail.com

Experiment No: 2**Problem Statement:**

Write a program to demonstrate the use of PreparedStatement and ResultSet interface.

Code:

```
import java.sql.*;

public class DatabaseDemo {

    public static void main(String[] args) {

        String url = "jdbc:mysql://localhost:3306/mydb";

        String username = "root";

        String password = "password";

        String createTableQuery = "CREATE TABLE IF NOT EXISTS employees (id INT, name VARCHAR(255), salary DOUBLE)";

        String insertQuery = "INSERT INTO employees (id, name, salary) VALUES (?, ?, ?)";

        String selectQuery = "SELECT * FROM employees";

        try (Connection conn = DriverManager.getConnection(url, username, password);

            Statement stmt = conn.createStatement();

            PreparedStatement pstmt = conn.prepareStatement(insertQuery);

            PreparedStatement pstmtSelect = conn.prepareStatement(selectQuery)) {

            stmt.executeUpdate(createTableQuery);

            // Insert data into the table using PreparedStatement

            pstmt.setInt(1, 1);

            pstmt.setString(2, "John Doe");

            pstmt.setDouble(3, 50000.0);

            pstmt.executeUpdate();

            pstmt.setInt(1, 2);
```

```
pstmt.setString(2, "Jane Doe");

pstmt.setDouble(3, 60000.0);

pstmt.executeUpdate();


// Retrieve data from the table using PreparedStatement

ResultSet resultSet = pstmtSelect.executeQuery();

// Print the retrieved data

while (resultSet.next()) {

    int id = resultSet.getInt("id");

    String name = resultSet.getString("name");

    double salary = resultSet.getDouble("salary");

    System.out.println("ID: " + id + ", Name: " + name + ", Salary: " + salary);

}

} catch (SQLException e) {

    System.out.println("Error: " + e.getMessage());

}

}
```

Output:

Employee Details

ID	Name	Salary
1	John Doe	50000.0
2	Jane Doe	60000.0

Experiment No: 3

Problem Statement:

Servlet Execution on tomcat

Code:

```
<!DOCTYPE html>

<html>

<head>

<meta charset="ISO-8859-1">

<title>Insert title here</title>

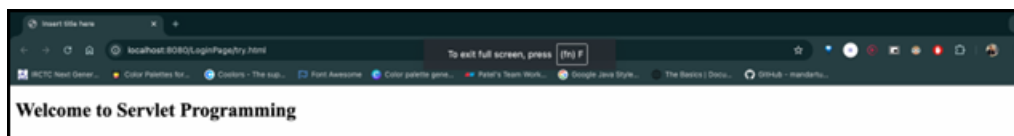
</head>

<body>

<h1>Welcome to Servlet Programming</h1>

</body>

</html>
```

Output:

Experiment No: 4**Problem Statement:**

A servlet program to print hello world.

Code:

```
import javax.servlet.*;

import java.io.*;

public class HelloWorldServlet extends HttpServlet {

    @Override

    public void doGet(HttpServletRequest request, HttpServletResponse response)

        throws ServletException, IOException {

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

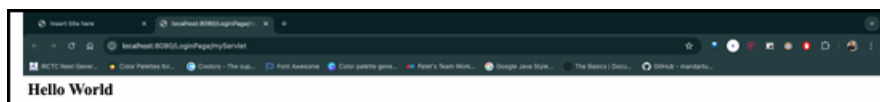
        out.println("<html><body>");

        out.println("<h1>Hello World!</h1>");

        out.println("</body></html>");

    }

}
```

Output:

Experiment No: 5_a**Problem Statement:**

A servlet program to display request details.

```
import javax.servlet.*;

import java.io.*;

public class RequestDetailsServlet extends HttpServlet {

    @Override

    public void doGet(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        out.println("<html><body>");out.println("<h1>Request Details</h1>");out.println("<p>Request
Method: " + request.getMethod() + "</p>");out.println("<p>Request URI: " + request.getRequestURI()
+ "</p>");

        out.println("<p>Request Protocol: " + request.getProtocol() + "</p>");

        out.println("<p>Request Query String: " + request.getQueryString() +
"</p>");out.println("<p>Request Remote Address: " + request.getRemoteAddr() +
"</p>");out.println("<p>Request Remote Host: " + request.getRemoteHost() + "</p>");

        out.println("</body></html>");

    }

}
```

Output:

Request Details

```
Request Method: GET
Request URI: /request-details
Request Protocol: HTTP/1.1
Request Query String: null
Request Remote Address: 127.0.0.1
Request Remote Host: localhost
```

Experiment No: 5_b**Problem Statement:****A servlet program to handle user form.****Code:****HTML File**

```
<!DOCTYPE html>

<html>

<head>

    <title>User Form</title>

</head>

<body>

<h1>User Form</h1>

<form action="TryCode" method="get">

<label for="name">Name:</label><br>

<input type="text" id="name" name="name"><br>

<label for="email">Email:</label><br>

<input type="email" id="email" name="email"><br>

<label for="phone">Phone:</label><br>

<input type="tel" id="phone" name="phone"><br>

<input type="submit" value="Submit">

    </form>

</body>

</html>

SERVLET FILE
package Serversite;
```

```
import jakarta.servlet.ServletException;

import jakarta.servlet.annotation.WebServlet;

import jakarta.servlet.http.HttpServlet;

import jakarta.servlet.http.HttpServletRequest;

import jakarta.servlet.http.HttpServletResponse;

import java.io.IOException;

import java.io.PrintWriter;

@WebServlet("/TryCode")

public class TryCode extends HttpServlet {

    protected void doGet(HttpServletRequest request,

        HttpServletResponse response) throws ServletException,

        IOException {

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        // Retrieving parameters from the request

        String name = request.getParameter("name");

        String email = request.getParameter("email");

        String phone = request.getParameter("phone");

        // Generating the response

        out.println("<html><body>");

        out.println("<h1>User Details:</h1>");

        out.println("<p>Name: " + name + "</p>");

        out.println("<p>Email: " + email + "</p>");
```

```

        out.println("<p>Phone: " + phone + "</p>");

        out.println("</body></html>");

        // Closing the PrintWriter

        out.close();

    }

}

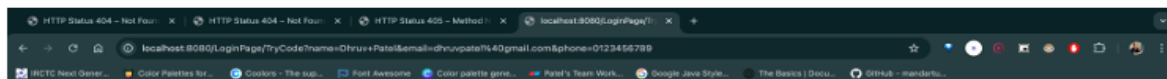
```

Output:



User Form

Name:
Dhruv Patel
Email:
dhruvpate1@gmail.com
Phone:
0123456789
Submit



User Details:

Name: Dhruv Patel
Email: dhruvpate1@gmail.com
Phone: 0123456789

Experiment No: 6_a**Problem Statement:**

A servlet program to create a cookie and display cookie.

Code:

```
package Serversite;

import java.io.IOException;

import java.io.PrintWriter;

import jakarta.servlet.ServletException;

import jakarta.servlet.annotation.WebServlet;

import jakarta.servlet.http.Cookie;

import jakarta.servlet.http.HttpServlet;

import jakarta.servlet.http.HttpServletRequest;

import jakarta.servlet.http.HttpServletResponse;

@WebServlet("/TryCode") // Replacing with the previous file name
and URL mapping

public class TryCode extends HttpServlet {

    private static final long serialVersionUID = 1L;

    protected void doGet(HttpServletRequest request,

        HttpServletResponse response) throws ServletException, IOException

    {

        'john_cena'

        Cookie cookie = new Cookie("username", "john_cena");

        cookie.setMaxAge(24 * 60 * 60); // Setting cookie to

        expire in 24 hours

        response.addCookie(cookie
```

```
response.setContentType("text/html");PrintWriter out = response.getWriter();

out.println("<html>");

out.println("<head>");

out.println("<title>Cookie Example</title>");

out.println("</head>");

out.println("<body>");

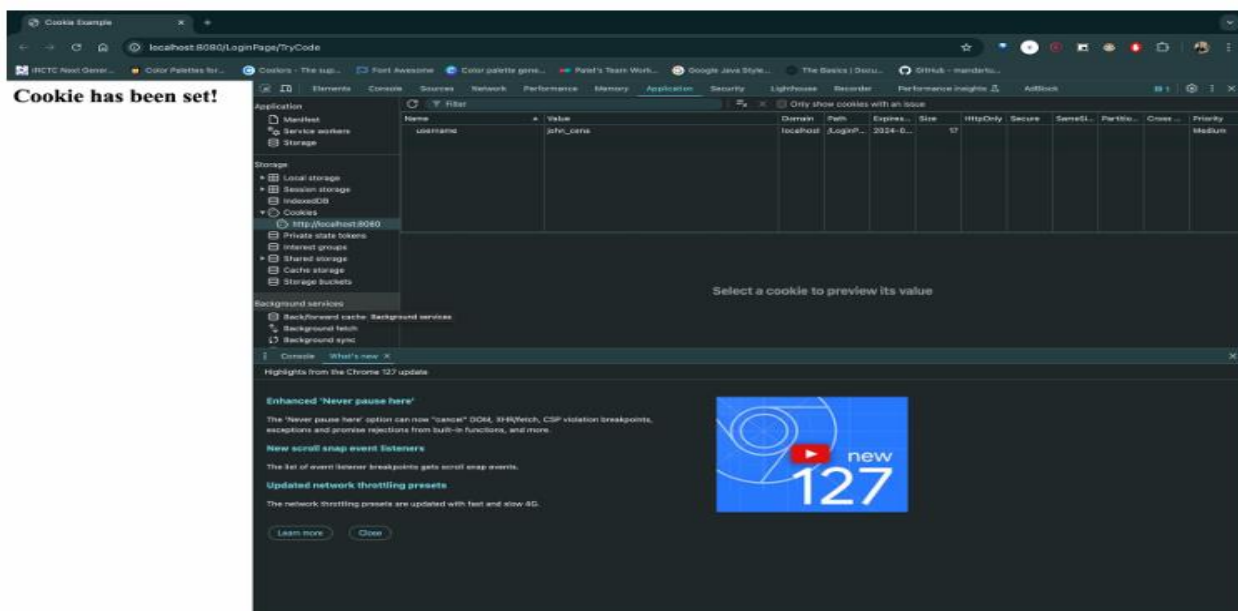
out.println("<h1>Cookie has been set!</h1>");

out.println("</body>");

out.println("</html>");

out.close(); // Closing the PrintWriter } }
```

Output:



Experiment No: 6_B**Problem Statement:**

A servlet program to display cookie

Code:

```
package Serversite;

import java.io.IOException;

import jakarta.servlet.ServletException;

import jakarta.servlet.annotation.WebServlet;

import jakarta.servlet.http.Cookie;

import jakarta.servlet.http.HttpServletRequest;

import jakarta.servlet.http.HttpServletResponse;

@WebServlet("/printCookies")

public class TryCode extends jakarta.servlet.http.HttpServlet {

    @Override

    protected void doGet(HttpServletRequest request,

        HttpServletResponse response)

        throws ServletException, IOException {

        // Set response content type

        response.setContentType("text/html");

        // Get cookies from the request

        Cookie[] cookies = request.getCookies();

        // Print cookies to the response

        response.getWriter().println("<html><body>");

        response.getWriter().println("<h1>Cookies:</h1>");
```

```
if (cookies != null && cookies.length > 0) {  
  
    for (Cookie cookie : cookies) {  
  
        response.getWriter().println("<p>Cookie Name: " +  
  
            cookie.getName() + "<br/>");  
  
        response.getWriter().println("Cookie Value: " +  
  
            cookie.getValue() + "</p>");  
  
    }  
  
} else {  
  
    response.getWriter().println("<p>No cookies found.</  
  
        p>");  
  
}  
  
response.getWriter().println("</body></html>");  
  
}  
  
}
```

Output:

Experiment No: 7**Problem Statement:**

A servlet program to do session tracking

Code:**HttpSessionEx1.java**

```
package mypackage;

import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;

@WebServlet("/HttpSessionEx1")

public class HttpSessionEx1 extends HttpServlet {

    private static final long serialVersionUID = 1L;

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");

        PrintWriter pw = response.getWriter();

        pw.print("<html><body>");
```

```
pw.print("<form name='frmlogin' method='post'>");

pw.print("<p>Username: <input type='text' name='username'/></p>");

pw.print("<p>Password: <input type='password' name='password'></p>");

pw.print("<input type='submit' value='Login'>");

pw.print("</form>");

pw.print("</body></html>");

}

@Override

protected void doPost(H pServletRequest request, H pServletResponse

    response) throws ServletException, IOException {

    try {

        response.setContentType("text/html");

        PrintWriter pw = response.getWriter();

        String name = request.getParameter("username");

        String password = request.getParameter("password");

        H pSession session = request.getSession();

        // Set the data to the session

        session.setAttribute("username", name);

        session.setAttribute("password", password);

        pw.print("<html><body>");

        pw.print("<a href='H pSessionEx1_1'>Click to view session info</a>");

        pw.print("</body></html>");

        pw.close();
```

```
        } catch (Exception exp) {  
            exp.printStackTrace();  
        }  
    }  
}
```

HttpSessionEx1_1.java

```
package mypackage;  
  
import java.util.Date;  
  
import java.io.IOException;  
  
import java.io.PrintWriter;  
  
import javax.servlet.ServletException;  
  
import javax.servlet.annotation.WebServlet;  
  
import javax.servlet.http.HttpServlet;  
  
import javax.servlet.http.HttpServletRequest;  
  
import javax.servlet.http.HttpServletResponse;  
  
import javax.servlet.http.HttpSession;  
  
@WebServlet("/HttpSessionEx1_1")  
  
public class HttpSessionEx1_1 extends HttpServlet {  
    private static final long serialVersionUID = 1L;  
  
    @Override  
  
    protected void doGet(HttpServletRequest request, HttpServletResponse  
        response) throws ServletException, IOException {  
        response.setContentType("text/html");  
    }  
}
```

```
PrintWriter pw = response.getWriter();

pw.write("<html><body>");

HttpSession session = request.getSession(false);

if (session == null || session.getAttribute("username") == null) {

    response.sendRedirect("HttpSessionEx1");

    return;

}

String myName = (String) session.getAttribute("username");

String myPass = (String) session.getAttribute("password");


pw.write("<p>Username: " + myName + "</p>");

pw.write("<p>Password: " + myPass + "</p>");

pw.write("<p>Creation Time of Session (ms): " +

    session.getCreationTime() + "</p>");

pw.write("<p>Creation Date: " + new Date(session.getCreationTime())

    + "</p>");

pw.write("<p>Last Accessed (ms): " + session.getLastAccessedTime()

    + "</p>");

pw.write("<p>Last Accessed Date: " + new

    Date(session.getLastAccessedTime()) + "</p>");

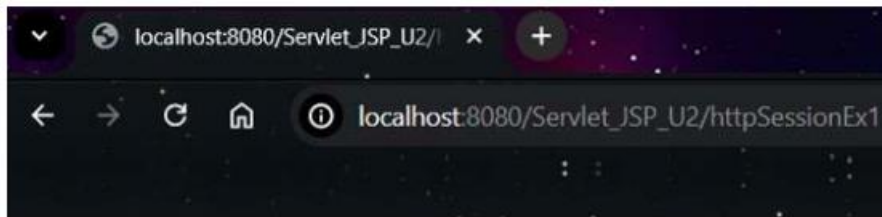
pw.write("</body></html>");

pw.close();
```

```
}
```

```
}
```

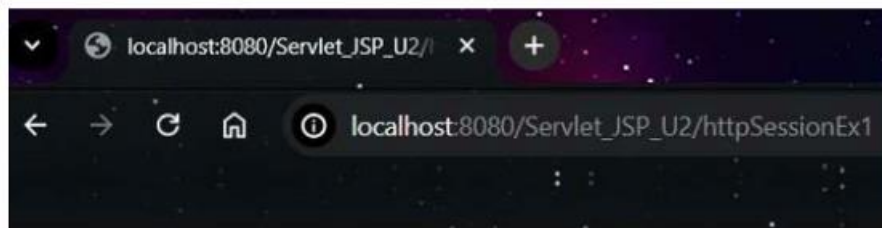
Output:



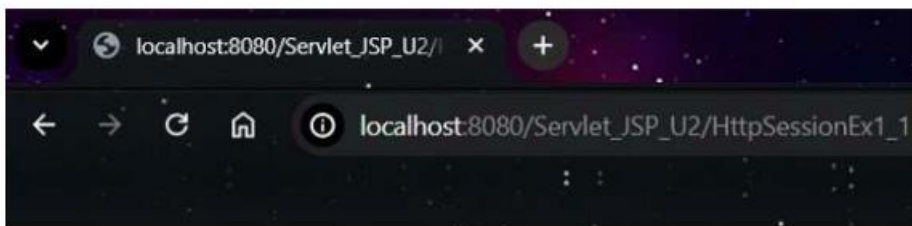
Username: Kush_Patel

Password:

Login



[Click to GO!](#)



Username :- Kush_Patel

Password :- 123456

Creation Time of Session (ms) :- 1721740856698

Creation Date :- Tue Jul 23 18:50:56 IST 2024

Last Accessed (ms) :- 1721741216279

Last Accessed Date :- Tue Jul 23 18:56:56 IST 2024

Experiment No: 8**Problem Statement:**

Write a program to implement chat Server using Server Socket and Socket class.

Code:**SERVER SOCKET**

```
import java.io.BufferedReader;

import java.io.IOException;

import java.io.InputStreamReader;

import java.io.PrintWriter;

import java.net.ServerSocket;

import java.net.Socket;

public class Server {

    private Socket socket;

    private ServerSocket server;

    private BufferedReader br;

    private PrintWriter out;

    public Server() {

        try {

            System.out.println("Server is ready to accept the connection!");

            server = new ServerSocket(8012);

            System.out.println("Waiting for client to connect...");

            socket = server.accept();

            System.out.println("Client connected!");

            // Initialize input and output streams
```



```
br = new BufferedReader(new InputStreamReader(socket.getInputStream()));

out = new PrintWriter(socket.getOutputStream(), true);

// Start reading and writing

startReading();

startWriting();

} catch (IOException ex) {

    ex.printStackTrace();

}

}

public void startReading() {

    System.out.println("Server - Reader starting");

    Runnable read = () -> {

        try {

            String msg;

            while ((msg = br.readLine()) != null) {

                if ("exit".equalsIgnoreCase(msg)) {

                    System.out.println("Client terminated!");

                    socket.close();

                    break;

                }

                System.out.println("Client: " + msg);

            }

        } catch (IOException e) {
```

```
        System.out.println("Connec on is closed");

    }

};

new Thread(read).start();

}

public void startWriting() {

    System.out.println("Server - Writer starting");

    Runnable write = () -> {

        try (BufferedReader bris = new BufferedReader(new InputStreamReader(System.in)))
        {

            String recdata;

            while ((recdata = bris.readLine()) != null) {

                out.println(recdata);

                if ("exit".equalsIgnoreCase(recdata)) {

                    socket.close();

                    break;

                }

            }

        } catch (IOException e) {

            System.out.println("Connec on is closed");

        }

    };

    new Thread(write).start();

}
```

```
}

public static void main(String[] args) {

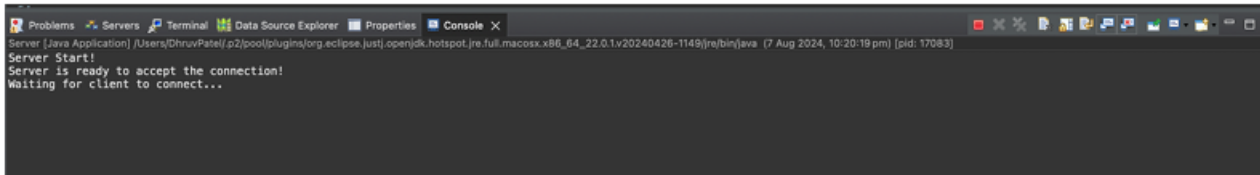
    System.out.println("Server Start!");

    new Server();

}

}
```

Output:



CLIENT SOCKET

Code:

```
import java.io.BufferedReader;

import java.io.IOException;

import java.io.InputStreamReader;

import java.io.PrintWriter;

import java.net.Socket;

public class Client {

    private Socket socket;

    private BufferedReader br;

    private PrintWriter out;

    public Client() {

        try {
```

```
// Connect to the server on port 8012

socket = new Socket("localhost", 8012);

System.out.println("Connected to server!");

// Initialize input and output streams

br = new BufferedReader(new InputStreamReader(socket.getInputStream()));

out = new PrintWriter(socket.getOutputStream(), true);

// Start reading and writing

startReading();

startWriting();

} catch (IOException ex) {

    ex.printStackTrace();

}

}

public void startReading() {

    System.out.println("Client - Reader starting");

    Runnable read = () -> {

        try {

            String msg;

            while ((msg = br.readLine()) != null) {

                System.out.println("Server: " + msg);

                if ("exit".equalsIgnoreCase(msg)) {

                    System.out.println("Server terminated!");

                    socket.close();

                }

            }

        } catch (IOException ex) {

            ex.printStackTrace();

        }

    };

    new Thread(read).start();

}
```

```
        break;
    }
}

} catch (IOException e) {
    System.out.println("Connection is closed");
}

};

new Thread(read).start();
}

public void startWriting() {
    System.out.println("Client - Writer starting");

    Runnable write = () -> {
        try (BufferedReader bris = new BufferedReader(new InputStreamReader(System.in)))
        {
            String msg;

            while ((msg = bris.readLine()) != null) {
                out.println(msg);

                if ("exit".equalsIgnoreCase(msg)) {
                    socket.close();

                    break;
                }
            }
        } catch (IOException e) {
```

```
        System.out.println("Connec on is closed");

    }

};

new Thread(write).start();

}

public static void main(String[] args) {

    System.out.println("Client Start!");

    new Client();

}

}
```

Output:



```
Server Start!
Server is ready to accept the connection!
Waiting for client to connect...
Client connected!
Server - Reader starting
Server - Writer starting
```

Experiment No: 9**Problem Statement:**

Write a Servlet program to send username and password using HTML forms and authenticate the user

Code:**HTML File**

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Login Form</title>

</head>

<body>

    <h1>Login Form</h1>

    <form action="TryCode" method="post"> <!-- Ensure this matches the servlet mapping -->

        <label for="username">Username:</label><br>

        <input type="text" id="username" name="username"><br><br>

        <label for="password">Password:</label><br>

        <input type="password" id="password" name="password"><br><br>

        <input type="submit" value="Login">

    </form>

</body>

</html>
```

Servlet File:

```
package Serversite;

import jakarta.servlet.ServletException;

import jakarta.servlet.annotation.WebServlet;

import jakarta.servlet.http.HttpServlet;

import jakarta.servlet.http.HttpServletRequest;

import jakarta.servlet.http.HttpServletResponse;

import java.io.IOException;

import java.io.PrintWriter;

@WebServlet("/TryCode") // Ensure this mapping matches the
form action URL

public class TryCode extends HttpServlet {

    private static final long serialVersionUID = 1L;

    protected void doPost(HttpServletRequest request,
HttpServletResponse response) throws ServletException,
IOException {

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        try {

            String username =
request.getParameter("username");

            String password =
request.getParameter("password");
```



```
if (username.equals("admin") &&  
password.equals("password")) {  
    out.println("Welcome, " + username + "!");  
    out.println("You have successfully logged  
in.");  
} else {  
    out.println("Invalid username or password.");  
    out.println("Please try again.");  
}  
} finally {  
    out.close(); // Ensure the PrintWriter is closed  
}
```

Output:



Experiment No: 10

Problem Statement:

JSP program to display hello world.

Code:

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>

<!DOCTYPE html>

<html>

<head>

    <meta charset="UTF-8">

    <title>Hello World JSP</title>

</head>

<body>

    <h1>Hello, World!</h1>

</body>

</html>
```

Output:

 Mero Share  Gmail  YouTube  Maps  Translate  Embassy of India, K...  Electronic Diversity...

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
```

Hello, World!

Experiment No: 11**Problem Statement:****JSP Program to demonstrate arithmetic operations.****Code:**

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Arithmetic Operations JSP</title>
</head>
<body>
    <h1>Arithmetic Operations</h1>
    <form action="arithmeticOperations.jsp" method="post">
        <label for="num1">Enter first number:</label>
        <input type="text" id="num1" name="num1" required>
        <br>
        <label for="num2">Enter second number:</label>
        <input type="text" id="num2" name="num2" required>
        <br>
        <input type="submit" value="Calculate">
    </form>
```

<%

String num1Str = request.getParameter("num1");

String num2Str = request.getParameter("num2");

if (num1Str != null && num2Str != null) {

try {

double num1 = Double.parseDouble(num1Str);

double num2 = Double.parseDouble(num2Str);

double sum = num1 + num2;

double difference = num1 - num2;

double product = num1 * num2;

double quotient = num2 != 0 ? num1 / num2 : Double.NaN; // Avoid division by zero

out.println("<h2>Results:</h2>");

out.println("<p>Sum: " + sum + "</p>");

out.println("<p>Difference: " + difference + "</p>");

out.println("<p>Product: " + product + "</p>");

if (!Double.isNaN(quotient)) {

out.println("<p>Quotient: " + quotient + "</p>");

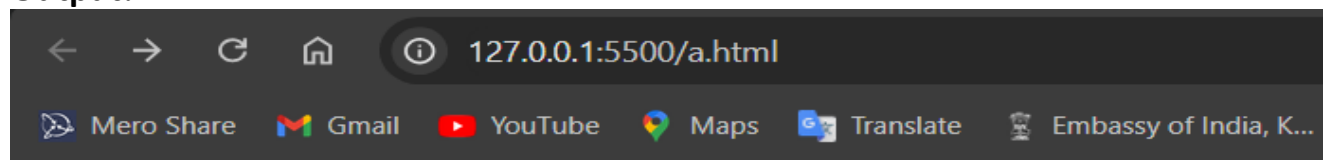
} else {

out.println("<p>Quotient: Cannot divide by zero</p>");

}

```
} catch (NumberFormatException e) {  
  
    out.println("<p>Invalid input. Please enter valid numbers.</p>");  
  
}  
  
}  
  
%>  
</body>  
</html>
```

Output:



Arithmetic Operations

Enter first number:
Enter second number:

Experiment No: 12**Problem Statement:**

JSP program to demonstrate jsp: forward action tag JSP program to request implicit object

Code:**Page1.jsp**

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>

<!DOCTYPE html>

<html>

<head>

    <title>Page 1</title>

</head>

<body>

    <h1>Welcome to Page 1</h1>

    <%

        // Setting an attribute in the request scope

        request.setAttribute("message", "Hello from Page 1!");

        // Forwarding the request to page2.jsp

        RequestDispatcher dispatcher = request.getRequestDispatcher("page2.jsp");

        dispatcher.forward(request, response);

    %>

</body>

</html>
```

Page2.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>

<!DOCTYPE html>

<html>

<head>

    <title>Page 2</title>

</head>

<body>

    <h1>Welcome to Page 2</h1>

    <p>Message from Page 1: <%= request.getAttribute("message") %></p>

</body>

</html>
```

Output:

Welcome to page 1
Welcome to page 2
Message from page 1

Experiment No: 13**Problem Statement:**

Create application to store the data in database to perform Hibernate CRUD operations

Main Class :

```
package com.example;

public class Main {

    public static void main(String[] args) {

        CRUDOperations crudOperations = new CRUDOperations();

        // Create an entity

        Entity entity = new Entity();

        entity.setName("Entity 1");

        entity.setDescription("This is entity 1");

        crudOperations.createEntity(entity);

        // Read an entity

        Entity readEntity = crudOperations.readEntity(1L);

        System.out.println("Read entity: " + readEntity.getName());

        // Update an entity

        entity.setDescription("This is updated entity 1");

        crudOperations.updateEntity(entity);

        // Delete an entity

        crudOperations.deleteEntity(1L);

    }

}
```


CURD Operation Class:

```
package com.example;

import org.hibernate.Session;

import org.hibernate.SessionFactory;

import org.hibernate.cfg.Configuration;

public class CRUDOperations {

    private SessionFactory sessionFactory;

    public CRUDOperations() {

        sessionFactory = new Configuration().configure().buildSessionFactory();

    }

    public void createEntity(Entity entity) {

        Session session = sessionFactory.getCurrentSession();

        session.beginTransaction();

        session.save(entity);

        session.getTransaction().commit();

    }

    public Entity readEntity(Long id) {

        Session session = sessionFactory.getCurrentSession();

        session.beginTransaction();

        Entity entity = session.get(Entity.class, id);

        session.getTransaction().commit();

        return entity;

    }

    public void updateEntity(Entity entity) {
```

```
Session session = sessionFactory.getCurrentSession();

session.beginTransaction();

session.update(entity);

session.getTransaction().commit();

}

public void deleteEntity(Long id) {

    Session session = sessionFactory.getCurrentSession();

    session.beginTransaction();

    Entity entity = session.get(Entity.class, id);

    session.delete(entity);

    session.getTransaction().commit();

}

}
```

OUTPUT:

```
1 Read entity: Entity 1
```

Experiment No: 14**Problem Statement:**

Create an application store the data in database to perform Spring CRUD operations.

Code:

```
package com.example.crudapp;

import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Service;
import org.springframework.web.bind.annotation.*;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

import org.springframework.beans.factory.annotation.Autowired;

import java.util.List;
import java.util.Optional;

@SpringBootApplication
public class CrudAppApplication {
```

```
public static void main(String[] args) {  
  
    SpringApplication.run(CrudAppApplication.class, args);  
  
}  
  
// Insert initial data when the application starts  
  
@Bean  
public CommandLineRunner dataInitializer(EmployeeRepository employeeRepository) {  
    return args -> {  
  
        employeeRepository.save(new Employee("John Doe", "Developer"));  
  
        employeeRepository.save(new Employee("Jane Smith", "Manager"));  
  
        employeeRepository.save(new Employee("Samuel Green", "Analyst"));  
  
        System.out.println("Data has been initialized");  
  
    };  
}  
}  
  
// Model Class  
  
@Entity  
class Employee {  
  
    @Id  
  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
  
    private Long id;
```

```
private String name;

private String role;

// Default constructor

public Employee() {

}

// Parameterized constructor

public Employee(String name, String role) {

    this.name = name;

    this.role = role;

}

// Getters and setters

public Long getId() {

    return id;

}

public void setId(Long id) {

    this.id = id;

}

public String getName() {

    return name;

}

public void setName(String name) {

    this.name = name;

}
```

```
public String getRole() {  
    return role;  
}  
  
public void setRole(String role) {  
    this.role = role;  
}  
}  
  
// Repository Interface  
interface EmployeeRepository extends JpaRepository<Employee, Long> {  
}  
  
// Service Layer  
@Service  
class EmployeeService {  
  
    @Autowired  
    private EmployeeRepository employeeRepository;  
  
    public List<Employee> getAllEmployees() {  
        return employeeRepository.findAll();  
    }  
  
    public Optional<Employee> getEmployeeById(Long id) {  
        return employeeRepository.findById(id);  
    }  
}
```

```
public Employee addEmployee(Employee employee) {  
    return employeeRepository.save(employee);  
}  
  
public Employee updateEmployee(Long id, Employee updatedEmployee) {  
    return employeeRepository.findById(id)  
        .map(employee -> {  
            employee.setName(updatedEmployee.getName());  
            employee.setRole(updatedEmployee.getRole());  
            return employeeRepository.save(employee);  
        }).orElseThrow(() -> new RuntimeException("Employee not found with id " + id));  
}  
  
public void deleteEmployee(Long id) {  
    employeeRepository.deleteById(id);  
}  
}  
  
// Controller Layer  
  
@RestController  
@RequestMapping("/api/employees")  
class EmployeeController {  
    @Autowired  
    private EmployeeService employeeService;  
  
    @GetMapping  
    public List<Employee> getAllEmployees() {
```

```
return employeeService.getAllEmployees();

}

@GetMapping("/{id}")
public Optional<Employee> getEmployeeById(@PathVariable Long id) {
    return employeeService.getEmployeeById(id);
}

@PostMapping
public Employee addEmployee(@RequestBody Employee employee) {
    return employeeService.addEmployee(employee);
}

@PutMapping("/{id}")
public Employee updateEmployee(@PathVariable Long id, @RequestBody Employee
updatedEmployee) {
    return employeeService.updateEmployee(id, updatedEmployee);
}

@DeleteMapping("/{id}")
public void deleteEmployee(@PathVariable Long id) {
    employeeService.deleteEmployee(id);
}
}
```

Output:
GET /api/employees


```
[  
  {  
    "id": 1,  
    "name": "John Doe",  
    "role": "Developer"  
  },  
  {  
    "id": 2,  
    "name": "Jane Smith",  
    "role": "Manager"  
  },  
  {  
    "id": 3,  
    "name": "Samuel Green",  
    "role": "Analyst"  
  }  
]
```

Experiment No: 15

Problem Statement:

Create a web application to store the data in database with spring boot.

Code:

src/main/resources/templates/employee_form.html

```
<!DOCTYPE html>

<html xmlns:th="http://www.thymeleaf.org">

<head>

    <title>Employee Form</title>

</head>

<body>

<h1>Employee Form</h1>

<form th:action="@{/employees}" th:object="${employee}" method="post">

    <input type="hidden" th:field="*{id}" />

    <label>Name: </label>

    <input type="text" th:field="*{name}" />

    <br />

    <label>Role: </label>

    <input type="text" th:field="*{role}" />

    <br />

    <button type="submit">Save</button>

</form>

<a th:href="@{/employees}">Back to List</a>

</body></html>
```

src/main/resources/templates/employees.html

```
<!DOCTYPE html>

<html xmlns:th="http://www.thymeleaf.org">

<head>

  <title>Employee List</title>

</head>

<body>

<h1>Employee List</h1>

<a th:href="@{/employees/new}">Add New Employee</a>

<table border="1">

  <thead>

    <tr>

      <th>ID</th>

      <th>Name</th>

      <th>Role</th>

      <th>Actions</th>

    </tr>

  </thead>

  <tbody>

    <tr th:each="employee : ${employees}">

      <td th:text="${employee.id}">ID</td>

      <td th:text="${employee.name}">Name</td>

      <td th:text="${employee.role}">Role</td>

      <td>
```

```
<a th:href="@{/employees/edit/{id}(id=${employee.id})}">Edit</a>  
<a th:href="@{/employees/delete/{id}(id=${employee.id})}">Delete</a>
```

```
</td>
```

```
</tr>
```

```
</tbody>
```

```
</table>
```

```
</body>
```

```
</html>
```

```
Webapp.java
```

```
package com.example.webapp;
```

```
import org.springframework.boot.CommandLineRunner;
```

```
import org.springframework.boot.SpringApplication;
```

```
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
import org.springframework.context.annotation.Bean;
```

```
import org.springframework.data.jpa.repository.JpaRepository;
```

```
import org.springframework.stereotype.Controller;
```

```
import org.springframework.ui.Model;
```

```
import org.springframework.web.bind.annotation.*;
```

```
import jakarta.persistence.Entity;
```

```
import jakarta.persistence.GeneratedValue;
```

```
import jakarta.persistence.GenerationType;
```

```
import jakarta.persistence.Id;
```

```
import org.springframework.beans.factory.annotation.Autowired;
```

```
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.Optional;

// Main Spring Boot application class
@SpringBootApplication
public class Application {

    public static void main(String[] args) {

        SpringApplication.run(Application.class, args);

    }

    @Bean
    public CommandLineRunner dataInitializer(EmployeeRepository employeeRepository) {

        return args -> {

            employeeRepository.save(new Employee("John Doe", "Developer"));
            employeeRepository.save(new Employee("Jane Smith", "Manager"));
            employeeRepository.save(new Employee("Samuel Green", "Analyst"));

            System.out.println("Data has been initialized");

        };

    }

}
```

```
// Entity class

@Entity

class Employee {

    @Id

    @GeneratedValue(strategy = GenerationType.IDENTITY)

    private Long id;

    private String name;

    private String role;

    public Employee() {}

    public Employee(String name, String role) {

        this.name = name;

        this.role = role;

    }

    public Long getId() { return id; }

    public void setId(Long id) { this.id = id; }

    public String getName() { return name; }

    public void setName(String name) { this.name = name; }

    public String getRole() { return role; }

    public void setRole(String role) { this.role = role; }

}

// Repository interface

interface EmployeeRepository extends JpaRepository<Employee, Long> {}

// Service class
```

@Service

```
class EmployeeService {  
  
    @Autowired  
    private EmployeeRepository employeeRepository;  
  
    public List<Employee> getAllEmployees() {  
        return employeeRepository.findAll();  
    }  
  
    public void saveEmployee(Employee employee) {  
        employeeRepository.save(employee);  
    }  
  
    public Optional<Employee> getEmployeeById(Long id) {  
        return employeeRepository.findById(id);  
    }  
  
    public void deleteEmployeeById(Long id) {  
        employeeRepository.deleteById(id);  
    }  
}
```

// Controller class

@Controller

```
class EmployeeController {  
  
    @Autowired  
    private EmployeeService employeeService;  
  
    @GetMapping("/employees")
```

```
public String viewEmployees(Model model) {  
    model.addAttribute("employees", employeeService.getAllEmployees());  
    return "employees";  
}  
  
@GetMapping("/employees/new")  
public String showNewEmployeeForm(Model model) {  
    model.addAttribute("employee", new Employee());  
    return "employee_form";  
}  
  
@PostMapping("/employees")  
public String saveEmployee(@ModelAttribute("employee") Employee employee) {  
    employeeService.saveEmployee(employee);  
    return "redirect:/employees";  
}  
  
@GetMapping("/employees/edit/{id}")  
public String showEditForm(@PathVariable Long id, Model model) {  
    model.addAttribute("employee", employeeService.getEmployeeById(id).orElse(null));  
    return "employee_form";  
}  
  
@GetMapping("/employees/delete/{id}")
```



```
public String deleteEmployee(@PathVariable Long id) {  
    employeeService.deleteEmployeeById(id);  
    return "redirect:/employees";  
}  
}
```

Output:

Employee Form

Name:
Role:

[Back to List](#)

Employee List

Add New Employee

ID	Name	Role	Actions
ID	Name	Role	Edit Delete